TrialMatch Transformer: Fine-Tuning, KV-Cache, Tiled Attention and Speculative Decoding for Clinical-Trial Criterion Screening

Muhammad Taha Irfan
Department of Computer Science (BSCS), FAST NUCES
Islamabad, Pakistan
tahairfan549@gmail.com

Code: https://github.com/Taha-tech05/TrialMatch-LM
Adapter: https://huggingface.co/Taha549/trialmatch-qwen3-4b-qlora
Demo video: https://youtu.be/e3O8ud70Kxo

***Abstract*—We build an educational system that reads a short patient note and one inclusion or exclusion criterion of a clinical trial and returns a criterion-level label with the numbers of the supporting note sentences. We fine-tune Qwen3-4B with QLoRA on the TrialGPT criterion annotations using a patient-grouped split of 37/8/8 patients. On the held-out test split (206 criterion rows, 8 patients, evaluated once) the fine-tuned model reaches accuracy 0.767 and macro-F1 0.726 over the classes present, against 0.665 and 0.523 for the base model and 0.626 and 0.272 for a majority-label baseline. The dominant error is answering "not excluded" where annotators judged "not enough information". We also implement a KV-cache decoder, an online-softmax tiled attention and greedy speculative decoding from scratch and measure them on a Tesla T4. The cache gives a 3.0 times speed-up on real validation prompts and up to 7.1 times at 1024 input tokens; tiled attention uses up to 180 times less peak memory than the reference at 4096 tokens; speculative decoding with an untuned Qwen3-0.6B draft gave no speed-up on real validation prompts. A browser application screens a pasted note against pasted criteria. This is an educational prototype, not a clinical enrollment tool.**

***Index Terms*—clinical trial matching, LoRA, KV cache, attention, speculative decoding**

I. Introduction

Matching a patient to a clinical trial means checking the patient against many inclusion and exclusion criteria. A useful assistant would, for each criterion, say whether the patient meets it and point to the note sentences that justify the answer. We study a small version of this problem with an open model that fits on a free Colab T4 GPU, and we use the project to implement and measure three inference techniques by hand: a key/value (KV) cache, a tiled attention with online softmax, and speculative decoding.

Problem statement. The input is a patient note whose sentences are numbered from 0, the type of one trial criterion (inclusion or exclusion) and the criterion text. The output is a JSON object with a *label* and an ascending list of *evidence ids*, the numbers of the note sentences that support the label (the list may be empty). Inclusion criteria take the labels *included, not included, not enough information (NEI), not applicable*; exclusion criteria take *excluded, not excluded, NEI, not applicable*. The task is scored per criterion against expert labels and expert evidence; a patient-level eligibility verdict is not part of the task and no gold verdicts exist.

Contributions: (i) a leakage-controlled data pipeline with a strict JSON output format, a parser and metrics that treat invalid outputs and empty evidence explicitly; (ii) a QLoRA fine-tuned Qwen3-4B evaluated on validation and, once, on test; (iii) tested hand-written cached decoding, tiled attention and speculative decoding, benchmarked on a T4, including a documented case of fp16 near-tie mismatches; (iv) a browser application whose demo mode never presents invented output as a model prediction. The system is an educational prototype and must not be used for real patient care or trial enrollment.

II. Related Work

Trial matching. The data are the criterion-level annotations released with TrialGPT [1], in which a large language model judges each criterion and cites supporting note sentences. The release also contains GPT-4 predictions, which we treat as forbidden inputs (Section III-A).

Models and fine-tuning. We use Qwen3 [2], LoRA adapters [3] and 4-bit base weights as in QLoRA [4].

Efficient inference. Attention follows [5]; the online softmax [6] and its blockwise use in FlashAttention [7] motivate our tiled version, which is plain PyTorch and therefore not fused. Grouped-query attention [8] and rotary embeddings [9] determine the size and indexing of the KV cache. Speculative decoding [10], [11] motivates our greedy draft-and-verify decoder.

III. Materials and Methods

A. Dataset and split

The annotation set has 1,015 criterion rows from 53 patients and 103 trials, with 5 to 43 rows per patient and exactly one note per patient (6.2 sentences on average, 3 to 14). There are 659 exclusion and 356 inclusion rows. The gold label is the expert label and the gold evidence the expert sentence list.

TABLE I
GOLD LABEL COUNTS (ROWS) PER SPLIT. SOURCE: DOCS/LABEL_DISTRIBUTION.MD.

	train	val	test	all
rows	603	206	206	1015
incl.: included	88	35	27	150
incl.: not included	13	3	7	23
incl.: NEI	109	27	38	174
incl.: not applicable	5	1	3	9
excl.: excluded	8	2	0	10
excl.: not excluded	292	101	91	484
excl.: NEI	65	26	28	119
excl.: not applicable	23	11	12	46

Leakage control. Four columns hold previous model outputs (`gpt4_explanation`, `explanation_correctness`, `gpt4_sentences`, `gpt4_eligibility`) and are removed from the processed files. The `training` column has unknown meaning and varies within 44 of 53 patients, so it is unused. Rows follow their patient: sorted patient ids are shuffled with seed 42 and the first 37, next 8 and last 8 patients form train, validation and test (603, 206, 206 rows). One row with a null criterion text (annotation 883, in train) is skipped.

Table I shows the main difficulty: the labels are very imbalanced. "Not excluded" alone is 484 of 659 exclusion rows (73.4%), "excluded" has 10 rows overall and *none* in test, and inclusion "not applicable" has 9 rows. About 60% of rows have empty gold evidence (0.601 in `docs/data_inspection.md`), so an empty evidence list is a valid and very common answer and is scored as such.

B. Prompt and output format

The model sees only the numbered note sentences, the criterion type and the criterion text, and answers with a JSON object {`"label": ..., "evidence_ids": [...]`} with ascending unique integer ids. No explanation is generated, because no gold explanations exist and the GPT-4 ones are forbidden. The parser never maps an invalid output to a valid label. The longest prompt plus target is 618 tokens with the Qwen chat template (thinking mode disabled), so a maximum length of 768 truncates nothing.

C. Model selection

Table II compares the candidates. We chose Qwen3-4B as the target and Qwen3-0.6B as the speculative draft. Both are open (Apache-2.0) and ungated, share one tokenizer (identical on a 200-string check, `docs/decisions.md`) and have 36 and 28 layers, a size ratio of $4.022/0.596 = 6.7$. The target's fp16 weights are estimated at 7.49 GiB; with the draft loaded the measured peak was about 9.08 GiB for speculative runs and 8.81 GiB for plain runs on the 14.56 GiB T4. Qwen3-1.7B was kept as a fallback if the 3-epoch projection exceeded 90 minutes; it was never needed. The Llama-3.2 pair (3B target, 1B draft) was rejected because both repositories were gated and returned HTTP 401, so nothing could be measured; its architecture figures in Table II were supplied from the model cards and not measured here. Two further reasons follow from them: the target-to-draft size ratio of $3.21/1.24 = 2.6$ is much smaller than 6.7 for Qwen3 and leaves less room for a speculative speed-up, and a long context window has no use when the longest prompt plus target is 618 tokens. The 0.75 B listed for the draft in the generated hub-metadata table differs from the 0.596 B reported by the loaded model; we use the loaded count. (The 4B model card likewise lists 4.0 B parameters of which 3.6 B are non-embedding, and the 0.6B, 1.7B and 4B models tie their embeddings.) The model card gives a native context of 32,768 tokens, extendable to 131,072 with YaRN, far above the 618 tokens needed here. Qwen3 is a decoder-only model with grouped-query attention (4 query heads per KV head for the 4B model), rotary embeddings, RMSNorm layers and an additional per-head RMSNorm on queries and keys (`q_norm`, `k_norm`) before the rotation, as read in `modeling_qwen3.py` of transformers 5.22.2. The embeddings are tied to the output layer. Our standalone attention functions do not implement the query/key normalisation and are not used inside the model; the cached and speculative decoders call the Hugging Face model, so its own attention runs.

D. System and model architecture

Fig. 1 shows how the parts connect. Everything on the lower row uses the single merged fp16 model, which makes the cached, uncached and speculative timings comparable and keeps quantisation overhead out of the numbers; the cost is that validation was also run on the 4-bit-base model and the two differ by three rows (Section V). The tiled attention study is deliberately separate: it benchmarks the attention operation with the 32/8 head layout of Qwen3-4B but does not replace the model's attention.

Fig. 2 (left) lists what the decoders have to respect: grouped-query attention makes the cache $2 \times 36 \times 8 \times 128 \times 2 = 147{,}456$ bytes per token in fp16, a quarter of what 32 KV heads would need, and rotary embeddings require the explicit position of every fed token. Fig. 2 (right) shows the speculative iteration; the crop is the step that keeps both caches consistent with the accepted tokens.

E. Fine-tuning

We fine-tune Qwen3-4B with QLoRA: NF4 4-bit base weights, fp16 compute, LoRA rank 16, alpha 32, dropout 0.05 on the attention and MLP projections only (embeddings are tied to the output layer, so the output layer is not adapted). Learning rate 2×10^{-4}, cosine schedule, 5% warm-up, 3 epochs, per-device batch 4 with 4 accumulation steps (effective batch 16), gradient checkpointing, seed 42. The loss covers the answer JSON and the end token only.

F. Evaluation protocol and metrics

All systems use greedy decoding with at most 64 new tokens and the same prompt. With gold label y_i and prediction $\hat{y}_i$ over N rows, accuracy is $\frac{1}{N}\sum_i [\hat{y}_i = y_i]$; an invalid output is a separate, always-wrong prediction. For a class c, $P_c =$

TABLE II

CANDIDATE MODELS. "LOADED" PARAMETER COUNTS COME FROM THE BENCHMARK ENVIRONMENT; THE GENERATED HUB-METADATA TABLE LISTS 0.75 B FOR THE DRAFT (SEE TEXT). LLAMA FIGURES WERE SUPPLIED BY THE AUTHOR FROM THE MODEL CARDS AND NOT MEASURED HERE. SOURCES: `DOCS/MODEL_FACTS.MD`, `DOCS/MEMORY_ESTIMATE.MD`, `RESULTS/BENCHMARKS/SPECULATIVE_20261108_113154.JSON`.

model	role	params (B)	layers	hidden	query / KV heads	fp16 weights (GiB)	status
Qwen3-4B	target	4.022 (loaded)	36	2560	32 / 8	7.49	chosen, fine-tuned
Qwen3-0.6B	draft	0.596 (loaded)	28	1024	16 / 8	1.11 (arithmetic)	chosen, untuned
Qwen3-1.7B	fallback	2.03 (hub metadata)	28	2048	16 / 8	3.78	never trained
Llama-3.2-3B	alt. target	3.21	28	3072	24 / 8	6.43 (arithmetic)	rejected: gated (401), not measured
Llama-3.2-1B	alt. draft	1.24	16	2048	32 / 8	2.48 (arithmetic)	rejected with its target

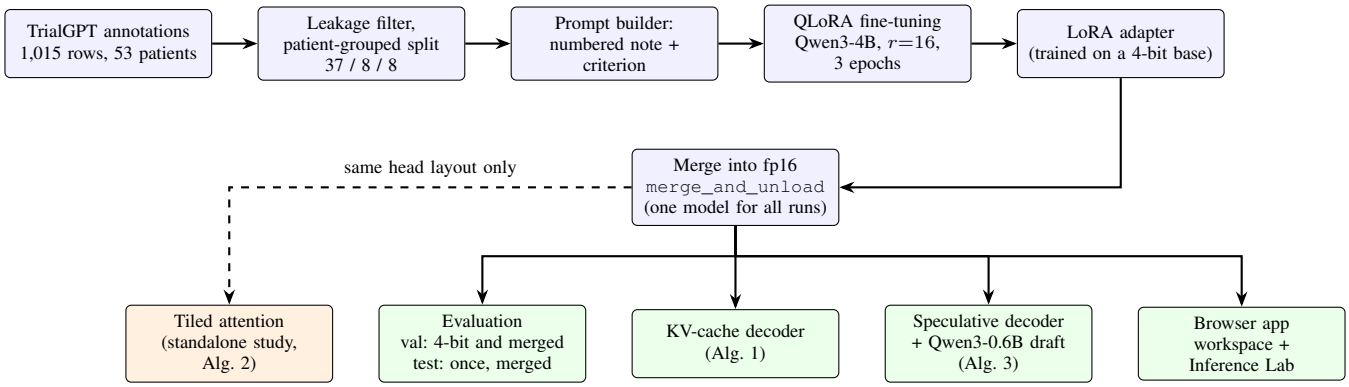


Fig. 1. System architecture. Data are split by patient, a LoRA adapter is trained on a 4-bit Qwen3-4B, merged into fp16 and used by the evaluation runner, the two hand-written decoders and the browser app; the tiled attention study runs on random tensors with the model's head layout and is not part of the model.

$\frac{TP_c}{TP_c+FP_c}$, $R_c = \frac{TP_c}{TP_c+FN_c}$ and $F1_c = \frac{2P_cR_c}{P_c+R_c}$, taken as 0 when undefined. Macro-F1 is averaged either over the classes present in the gold labels, $\mathcal{C}_{\text{gold}}$, or over the full label set $\mathcal{C}$ of the scope:

$$\text{F1}_{\text{present}} = \frac{1}{|\mathcal{C}_{\text{gold}}|} \sum_{c \in \mathcal{C}_{\text{gold}}} F1_c, \quad \text{F1}_{\text{full}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F1_c. \quad (1)$$

The two differ when a class has no gold rows. For evidence, with gold sentence set E and predicted set $\hat{E}$ per row, $P = \frac{|E \cap \hat{E}|}{|\hat{E}|}$, $R = \frac{|E \cap \hat{E}|}{|E|}$ and $F1$ as above, with the rule that $E = \hat{E} = \emptyset$ scores 1, exactly one of them empty scores 0, and an invalid output scores 0. Micro-F1 applies the same formula to the pooled sentence counts, macro-F1 averages the per-row values; each is reported over all rows (incl.) and over rows with non-empty gold evidence (excl.). For the decoders, the speed-up is $S = T_{\text{plain}}/T_{\text{method}}$ (medians of repeated runs) and the speculative acceptance rate is accepted candidates divided by proposed candidates (`src/trialmatch/inference/speculative/stats.py`). The baseline predicts the training-majority label per criterion type and always-empty evidence. The test split needs an explicit confirmation flag and a single run; it was evaluated once with the merged model.

G. Merged fp16 model

The adapter was trained on a 4-bit base and is published on the Hugging Face Hub (title block); the merged model (about 8 GB) is not committed. For all inference benchmarks we merge it into fp16 weights (`merge_and_unload`), giving one model of 4,022,468,096 parameters. On four validation prompts the merged and unmerged logits differ by at most 0.048 and the top-1 token agrees on all four.

Algorithm 1 KV-cache greedy decoding

Require: prompt $x_{1..T}$, budget N

prefill: logits, cache $\leftarrow$ model(x); emit $t_1 = \arg\max$; $pos \leftarrow T$

for $i = 2$ **to** N **(stop at EOS) do**

  mask $\leftarrow$ [mask, 1] {length $|cache| + 1$}

  logits, cache $\leftarrow$ model(t_{i-1}, $position_ids$=pos, $cache_position$=$|cache|$, cache)

  emit $t_i = \arg\max$ logits; $pos \leftarrow pos + 1$

end for

H. Attention and decoding algorithms

KV-cache decoding. `cached_greedy_decode` (`src/trialmatch/inference/kv_cache/cached_decode.py`) is Algorithm 1: one prefill over the prompt, then one-token steps that pass the cache, the true position and a mask that grows by one column. The reference decoder runs the full forward pass at every step.

Tiled attention. `tiled_attention` (Algorithm 2) keeps a running maximum m, normaliser l and output o per query block; on a new key block it rescales by $\alpha = \exp(m - m')$, so o/l is the softmax attention over the keys seen so far and exact after the last block. Key blocks above the causal diagonal are skipped. Query i sees key j iff $j \le i + (S_k - S_q)$; arithmetic is fp32 with the output cast back; grouped-query attention repeats each KV head for its query group. The reference materialises

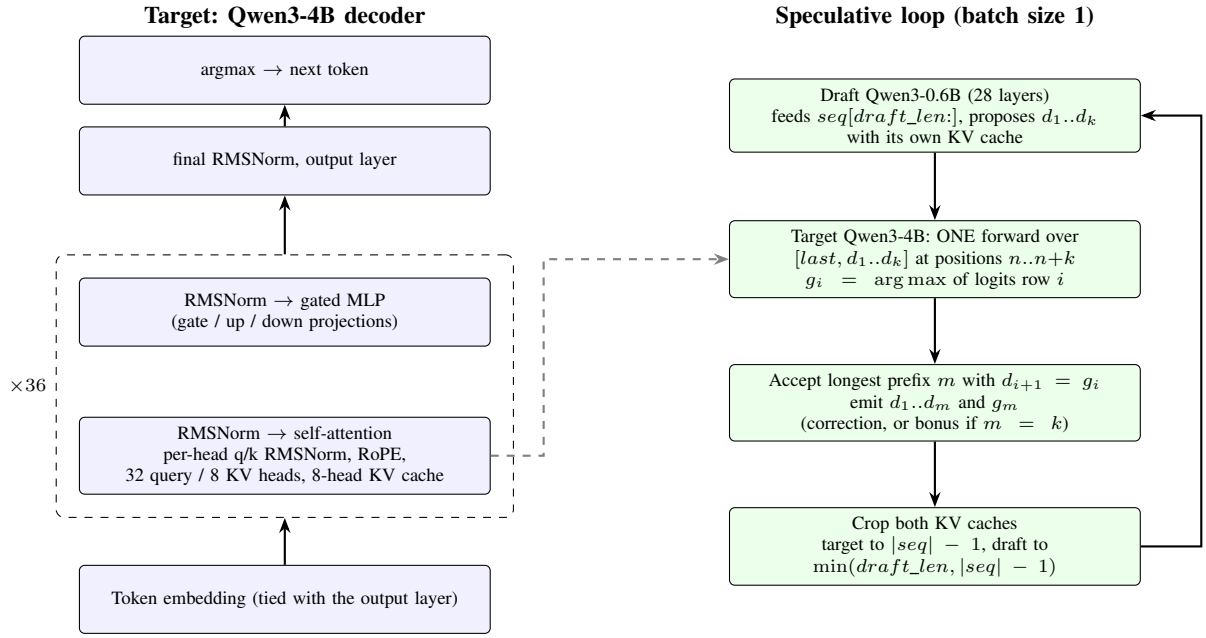


Fig. 2. Left: the Qwen3-4B decoder (36 layers) as read from `modeling_qwen3.py` and `docs/model_facts.md`. Right: one iteration of our speculative decoder (`src/trialmatch/inference/speculative/speculative_decode.py`); the dashed line marks that the verify step is the target model's ordinary forward pass with its KV cache.

Algorithm 2 Tiled online-softmax attention (one query block q)

$m \leftarrow -\infty,\ l \leftarrow 0,\ o \leftarrow 0$
for each key block (K_j, V_j) not above the causal diagonal **do**
  $s \leftarrow qK_j^\top / \sqrt{d}$ {mask on the diagonal block}
  $m' \leftarrow \max(m, \mathrm{rowmax}(s))$; $\alpha \leftarrow e^{m-m'}$; $p \leftarrow e^{s-m'}$
  $l \leftarrow \alpha l + \mathrm{rowsum}(p)$; $o \leftarrow \alpha o + pV_j$; $m \leftarrow m'$
end for
return o/l

Algorithm 3 Greedy speculative decoding (one iteration)

Require: seq (target cache $= |seq| - 1$), k, remaining budget r
$k' \leftarrow \min(k, r - 1)$
draft feeds $seq[draft_len:]$, then proposes $d_1..d_{k'}$ greedily
$g \leftarrow \arg\max$ target$([seq_{-1}, d_1..d_{k'}])$ {one forward}
$m \leftarrow$ longest prefix with $d_{i+1} = g_i$
append $d_1..d_m$ and g_m to seq (cut at EOS or budget)
crop target cache to $|seq| - 1$, draft cache to $\min(draft_len, |seq| - 1)$

the full score matrix; torch SDPA backends are comparison only.

Speculative decoding. `speculative_greedy_decode` (batch size 1, Algorithm 3) lets the draft propose k tokens, verifies them with one target forward over the last token plus the candidates, accepts the longest matching prefix, emits the target's own token as correction (or bonus when all k match) and crops both caches. Its output equals plain greedy decoding of the target up to floating-point ties.

Algorithm 1 is the baseline for the speed-ups in Section V-C; Algorithm 2 is evaluated on its own in Section V-D; Algorithm 3 yields between 1 and $k+1$ tokens per target forward, the quantity reported as "tokens per verify" in Section V-E.

I. Application workflow

A FastAPI backend with one static page offers a screening workspace and an Inference Lab (Fig. 3). A pasted note without numbering is split into sentences and numbered in the dataset format, so the model sees the same layout as in training; criteria are parsed from "Inclusion Criteria:" and "Exclusion Criteria:" sections. Each card shows a badge chosen by *outcome* (for exclusion criteria "not excluded" is the favourable one), the cited sentences, a template explanation and flags for missing information or invalid output. The rule is: any "excluded" or "not included" gives "likely ineligible (preliminary)"; otherwise any NEI, invalid or errored criterion gives "undetermined"; otherwise "likely eligible (preliminary)"; "not applicable" criteria are ignored. The app always states that this outcome is not a ground-truth label. Demo mode answers only from cached real validation predictions and otherwise says it cannot predict.

IV. Experimental Setup

Training and benchmarks ran on a Colab Tesla T4 (14.56 GiB) with Python 3.13, torch 2.14.1+cu130, transformers 5.22.1, peft 0.21.2 and bitsandbytes 0.50.2. Training took about 33 minutes (114 optimiser steps; log from 10:40:44 to 11:13:53). *KV cache:* synthetic inputs of 128, 256, 512, 1024 tokens with 16, 64, 128 new tokens (EOS disabled) and 16 real validation prompts, 1 warm-up and 5 repeats, decoders interleaved. *Attention:* random fp16 inputs, 32 query heads, 8 KV heads, head dimension 128, causal, lengths 128 to 4096,

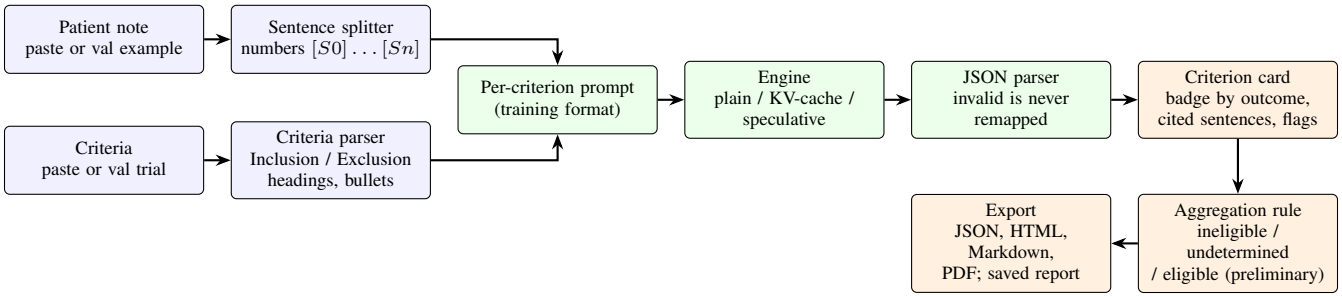


Fig. 3. Application workflow of the screening workspace. Notes and criteria are numbered or parsed, each criterion is screened separately with the selected engine, the JSON answer is parsed, a card is built from the label and the cited sentences (template explanation, no model call), and a rule aggregates the cards into a preliminary outcome.

TABLE III
VALIDATION SPLIT (206 ROWS). EVID. F1 IS MICRO-F1 OVER ALL ROWS (INCL.) AND OVER ROWS WITH NON-EMPTY GOLD EVIDENCE (EXCL.). FINE-TUNED = 4-BIT-BASE MODEL. SOURCE: `RESULTS/METRICS/VAL_COMPARISON.MD`.

system	acc.	macro-F1	ev. incl.	ev. excl.	invalid
majority baseline	0.621	0.215	0.000	0.000	0.000
base Qwen3-4B	0.626	0.312	0.359	0.519	0.000
fine-tuned	0.748	0.549	0.529	0.637	0.000

TABLE IV
TEST SPLIT (206 ROWS, 8 PATIENTS, ONE EVALUATION). F1 PRES. / F1 FULL ARE MACRO-F1 OVER PRESENT AND OVER ALL CLASSES; THEY DIFFER BECAUSE "EXCLUDED" HAS NO GOLD TEST ROWS. FINE-TUNED = MERGED FP16 MODEL. SOURCE: `RESULTS/METRICS/TEST_COMPARISON.MD`.

system	acc.	F1 pres.	F1 full	ev. incl.	ev. excl.	inv.
majority baseline	0.626	0.272	0.226	0.000	0.000	0.000
base Qwen3-4B	0.665	0.523	0.435	0.376	0.550	0.000
fine-tuned (merged)	0.767	0.726	0.605	0.578	0.663	0.000

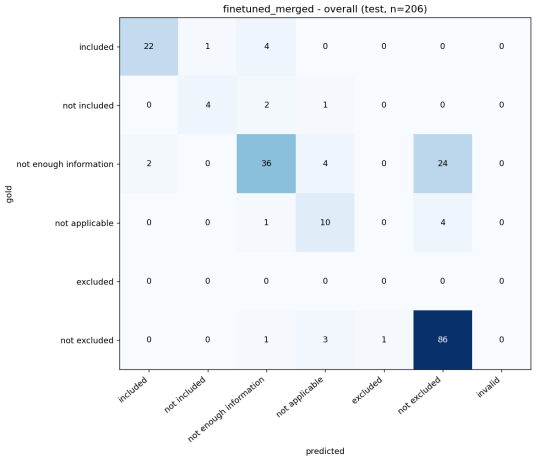


Fig. 4. Confusion matrix of the fine-tuned merged model on test (rows gold, columns predicted). Source: `results/figures/confusion_finetuned_merged_overall_test.png`.

block sizes 32 to 256, 2 warm-ups, 10 repeats, error against an fp32 reference. *Speculative:* untuned Qwen3-0.6B draft, $k \in \{1, 2, 4, 8\}$, synthetic inputs of 128, 256, 512 tokens with 16/64/128 new tokens and 8 validation prompts, 3 repeats. Timings are medians unless stated; attention memory is the peak above the run's starting level. The application screenshots were taken in real mode on the same kind of Colab T4 (header "cuda, float16").

V. RESULTS AND DISCUSSION

A. Classification and evidence

Table III shows that the majority baseline reaches accuracy 0.621, almost the base model's 0.626, because "not excluded" dominates the exclusion rows; macro-F1 separates them (0.215 vs 0.312). Fine-tuning raises accuracy by 0.122 and macro-F1 by 0.237 over the base model, and evidence micro-F1 from 0.359 to 0.529. No system produced an invalid output on validation.

On test (Table IV) the ordering is the same: accuracy 0.767 against 0.665 for the base model (+0.102) and 0.626 for the baseline; macro-F1 over present classes (Eq. 1) is 0.726 against 0.523 and 0.272. Counting the absent "excluded" class as F1 of 0 lowers the fine-tuned macro-F1 to 0.605, so both numbers are reported. Evidence micro-F1 rises from 0.376 to 0.578 over all rows and from 0.550 to 0.663 on rows with non-empty gold evidence. By criterion type, the fine-tuned model has accuracy 0.813 on the 75 inclusion rows and 0.740 on the 131 exclusion rows, and evidence micro-F1 over all rows of 0.676 for inclusion but only 0.314 for exclusion. Validation and test use different model variants: the merged model scored 0.733 on validation against 0.748, three rows of 206.

Fig. 4 locates the dominant error. Of 66 gold-NEI rows (36 + 24 + 4 + 2), 36 are correct but 24 are predicted "not excluded" (4 "not applicable", 2 "included"); in contrast 86 of 91 gold "not excluded" rows are correct. The per-class table agrees: NEI on exclusion criteria has precision 0.75 but recall 0.11 on test. The model seems to default to "not excluded" when a criterion is not discussed in the note, which is the training majority (75.3% of train exclusion rows, `docs/label_distribution.md`).

Fig. 5 shows the training loss falling from 0.473 at step 1 to 0.018 at step 114 while the validation loss stays near 0.12

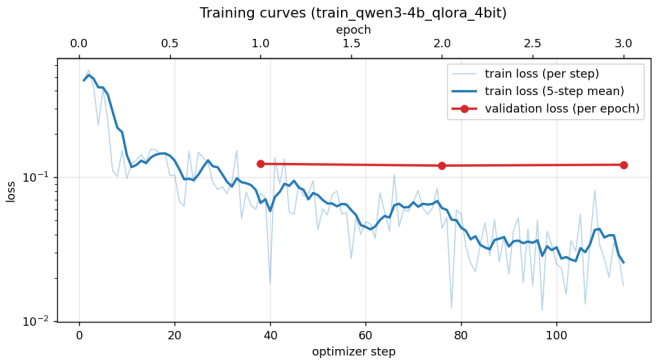


Fig. 5. Training and validation loss. Source: `results/figures/training_curves.png`.

TABLE V
ROWS PER ERROR CATEGORY (A ROW CAN HAVE SEVERAL). VAL: 4-BIT-BASE MODEL, TEST: MERGED MODEL. SOURCES: `RESULTS/ERROR_ANALYSIS/VAL_FINETUNED_ERROR_SUMMARY.CSV` AND COUNTS FROM `RESULTS/ERROR_ANALYSIS/TEST_FINETUNED_MERGED_ERRORS.CSV`.

category	val	test
NEI gold, decided prediction	18	26
decided gold, NEI prediction	16	7
applicable vs. not applicable	16	13
polarity swap	2	2
evidence wrong / missing / extra	7 / 27 / 32	3 / 22 / 20
cites consent sentence	4	1
any error (of 206)	92	81

(0.124, 0.121, 0.123 after epochs 1 to 3). A flat validation loss with a falling training loss is consistent with overfitting a 603-row training set; we ran no ablation (fewer epochs, other ranks) to confirm this.

B. Error analysis

Table V counts the categories: 92 of 206 validation rows and 81 of 206 test rows have at least one error, but label errors alone are 52 (val) and 48 (test) rows, matching the accuracies 0.748 and 0.767; the rest are right labels with a wrong evidence set. The largest label category on test is NEI answered with a decided label (26 rows), and 18 of those come from one patient, who accounts for 25 of the 81 test errors. Table VI shows representative cases. The exclusion NEI misses (val 8 and 19, test 392 and 193) share one pattern: the criterion is not discussed in the note and the model answers with the majority label; the two polarity swaps on the rare "excluded" class (val 263, test 184) and the inclusion swaps (val 935, test 185) need a small inference the model does not make; and the consent sentence is cited in 4 validation rows and 1 test row (val 375, test 381), the planned check of the boilerplate sentence. Evidence errors with a correct label (val 13 and 5) are mostly over- or under-citation. All causes are hypotheses.

Observations in the application. Using the app we observed a myocardial infarction *three months* ago labelled "not excluded" for a criterion excluding a myocardial infarction

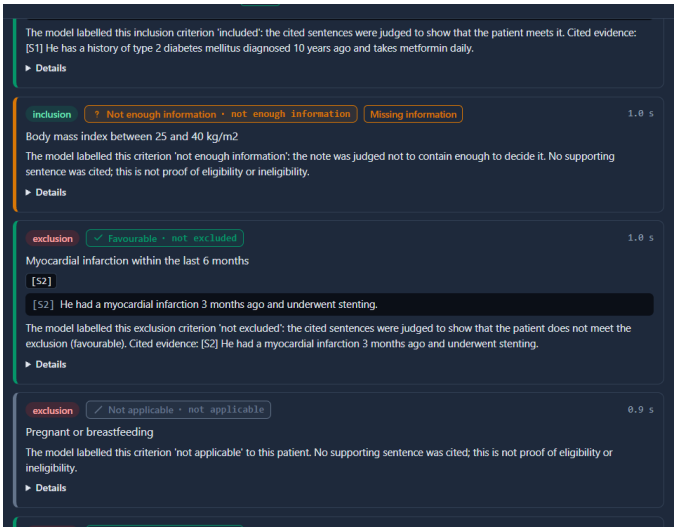


Fig. 6. Application cards for the myocardial infarction case: the exclusion criterion "Myocardial infarction within the last 6 months" is labelled "not excluded" and cites S2 ("3 months ago"). Cropped screenshot from the author's session of 2026-11-10 (`demo/11_mi_3_months_not_excluded.png`).

within six months, and a criterion never mentioned in the note labelled "not excluded". Fig. 6 shows the first case: the exclusion criterion "Myocardial infarction within the last 6 months" is labelled "not excluded" (favourable) while the cited sentence S2 reads "He had a myocardial infarction 3 months ago and underwent stenting", so the model cites exactly the sentence that contradicts its label (latency 1.0 s, read from the screenshot). Had the card said "excluded", the rule would have returned "likely ineligible" instead of "undetermined" (Fig. 10(b) shows 0 decisive-negative and 8 favourable criteria). The same screenshot shows the body-mass-index inclusion criterion labelled NEI with the missing-information flag and "Pregnant or breastfeeding" labelled not applicable for a male patient, both plausible. The second observation has no screenshot. Both observations point at the same weakness: for exclusion criteria the model treats missing information as absence, and it does not compare a time window in the criterion with a time in the note.

C. KV cache

Table VII and Fig. 7 show that the cached decode time per token is roughly constant (78 to 96 ms at all input lengths) while the uncached decoder grows from about 93 ms per token at 128 input tokens to about 635 ms at 1024, since it reprocesses the whole sequence at each step. The speed-up therefore grows from 1.43 times at 128 input tokens to 7.10 times at 1024, and is 3.02 times (range 2.20 to 3.28) over the real validation prompts (317 to 338 input tokens). In the first row the cached decoder was not faster (0.93 times); the repeat spread (cached total standard deviation 0.187 s) exceeds that gap, so we regard it as noise. The cached decoder's peak memory above the starting level is 246.5 MiB at 1024 input tokens against 81.9 MiB uncached, on top of about 8.1 to 8.3

TABLE VI

TWELVE DRAFTED ERROR CASES WITH HEDGED CAUSES (HYPOTHESES; THE MODEL CANNOT BE INSPECTED). PRED. = PREDICTED LABEL; [..] = CITED SENTENCE IDS. SOURCE: RESULTS/ERROR_ANALYSIS/VAL_FINETUNED_ERRORS.CSV AND TEST_FINETUNED_MERGED_ERRORS.CSV, COLUMN MY_NOTES; FULL TEXT IN DOCS/ERROR_ANALYSIS.MD.

split	id	type	gold → pred.	category	likely cause
val	8	excl.	NEI [0-2] → not excluded [0-7]	NEI→decided; evidence extra	"Unstable angina" is not stated; a missing diagnosis is probably read as "not excluded" and the whole note is cited
val	19	excl.	NEI [] → not excluded []	NEI→decided	criterion probably never mentioned; default to the majority label
val	263	excl.	excluded [5] → not excluded [1,4]	polarity; evidence wrong	"vesicular mole" not linked to "uterine pathology"; rare class (8 train rows)
val	935	incl.	not included [3] → included [3]	polarity	same sentence cited; likely matched "culture of the sputum" to "culture positive" without checking the organism
val	375	incl.	NEI [0,1] → included [0,1,6]	NEI→decided; consent cited	boilerplate consent sentence probably taken as support
val	13	excl.	not excluded [4] → not excluded []	evidence missing	negation sentence skipped; 75.6% of "not excluded" rows have empty gold evidence
val	5	excl.	not excluded [] → not excluded [3]	evidence extra	a topically related sentence is over-cited
test	392	excl.	NEI [] → not excluded []	NEI→decided	one of 18 such rows in one patient; same default for never-mentioned criteria
test	185	incl.	included [2,3] → not included []	polarity; evidence missing	shock not inferred from vital signs (BP 80/60, tachycardia)
test	184	excl.	not excluded [] → excluded [0-2]	polarity; evidence extra	vague criterion ("other inappropriate situation") mapped to the acute complication
test	381	incl.	included [0] → included [4]	evidence wrong; consent cited	lexical overlap on "informed consent" with the boilerplate sentence
test	193	excl.	NEI [0-4] → not excluded []	NEI→decided; evidence missing	"moribund" not stated; model judged nothing indicates it

TABLE VII

CACHED VS. UNCACHED GREEDY DECODING ON THE T4: MEDIAN TOTAL TIME (S) AND SPEED-UP (OUR DIVISION OF MEDIANS). SOURCE: RESULTS/BENCHMARKS/KV_CACHE_20261107_192136.JSON.

input	new	cached	uncached	speed-up
128	16	1.573	1.470	0.93
128	128	11.649	16.646	1.43
256	128	11.305	25.972	2.30
512	128	10.125	42.983	4.25
1024	16	1.680	9.745	5.80
1024	128	11.420	81.125	7.10
val (16)	15–38	1.219	3.025	3.02

TABLE VIII

ATTENTION ON THE T4 (FP16, BATCH 1, 32 QUERY / 8 KV HEADS, HEAD DIM 128): MEDIAN MS / PEAK MIB ABOVE START. SOURCE: RESULTS/BENCHMARKS/ATTENTION_20261107_193238.JSON.

seq	reference	tiled 32	tiled 256	SDPA eff.
128	1.75 / 15.0	7.48 / 4.8	1.77 / 19.1	0.83 / 3.0
512	4.89 / 132	77.3 / 7.8	5.00 / 52.2	1.08 / 12.0
1024	16.9 / 457	309 / 11.8	13.6 / 56.2	2.23 / 24.0
2048	50.4 / 1685	1148 / 19.8	33.6 / 64.2	2.78 / 48.0
4096	197 / 6449	4515 / 35.8	114 / 80.2	8.28 / 96.0

GB of weights; the cache itself is about 0.141 GiB at 1024 tokens by the formula.

fp16 near-tie. One of the 12 synthetic cells (256 input, 128 new tokens) produced different tokens: the first mismatch is at step 103 with a maximum logit difference of 0.0293. All other synthetic cells and all 16 validation prompts matched. We read this as a rounding effect (prefill and single-token steps use different matrix shapes) rather than a logic error, since the same decoders agree exactly in fp32 tests on a tiny model; we did not isolate the cause, and the report does not record the top-1/top-2 margin at that step. In contrast, the negative test that gives every decode step position 0 (tiny random model) diverges at the first decode step with a logit difference of 3.505 (pytest output supplied by the author, not stored in the repo), so a wrong position is far from a near-tie.

D. Tiled attention

Table VIII and Fig. 8 show the expected memory behaviour: the reference's peak memory roughly quadruples per doubling at the long end (1684.5 to 6449.1 MiB from 2048 to 4096), whereas the tiled version stays between 35.8 and 80.2 MiB at 4096 depending on block size, 80 to 180 times smaller. The tiled results are exact to fp16 rounding: the reference, every tiled variant and the math SDPA backend share the same maximum absolute error against the fp32 reference (0.000921 at 128 tokens, 0.000966 at 4096), the memory-efficient SDPA backend has at most 0.00120, and all are within the 0.005 tolerance with no mismatch recorded. Speed is different. With block 32 the plain-PyTorch loop is 22.9 times slower than the reference at 4096 tokens (4514.71 vs 197.06 ms), and time falls about 3 to 4 times per doubling of the block size, which fits the number of Python-level iterations and kernel launches; with block 256 it is 1.7 times faster than the reference at 4096 tokens (113.85 ms) but slightly slower at short lengths (2.60 vs 2.10 ms at 256). The memory-efficient SDPA kernel is fastest everywhere (8.28 ms at 4096, 23.8 times faster than our reference); the flash backend was reported unavailable at every length ("No available kernel"), for a reason the repo does not record. A PyTorch loop can match the memory advantage of FlashAttention-style tiling but not the speed of a fused kernel. The SDPA calls use fp16 arithmetic while ours accumulate in fp32, so the speed comparison is not like for like.

E. Speculative decoding

Table IX and Fig. 9 show that speculative decoding did not speed up real validation prompts. The acceptance rate falls from 0.974 at $k = 1$ to 0.500 at $k = 8$ and tokens per verify rise only from 1.97 to 4.67, while the draft's share of time

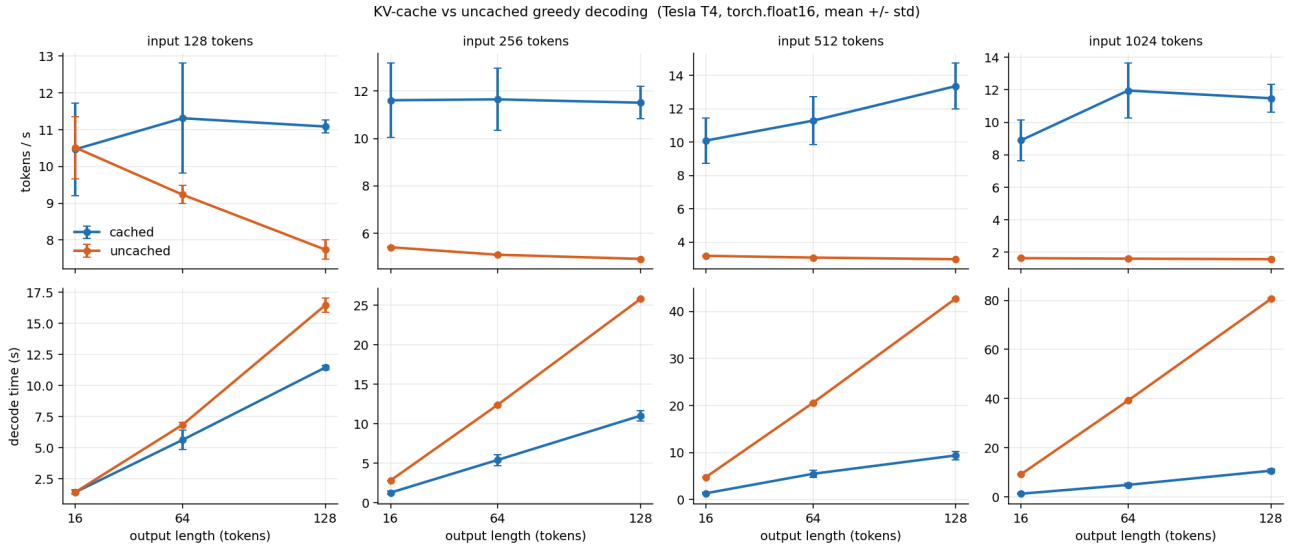


Fig. 7. Tokens per second and decode time, cached vs. uncached, by input length. Source: `results/benchmarks/kv_cache_20261107_192136.png`.

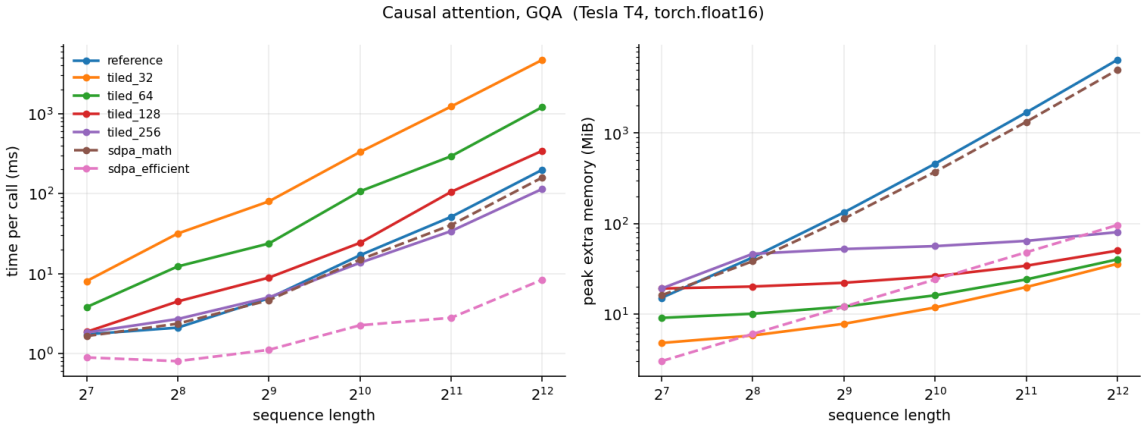


Fig. 8. Attention time and peak extra memory vs. sequence length (log scale). Source: `results/benchmarks/attention_20261107_193238.png`.

TABLE IX
SPECULATIVE DECODING ON THE 8 REAL VALIDATION PROMPTS (UNTUNED QWEN3-0.6B DRAFT): MEDIANS OVER PROMPTS; SPEED-UP = PLAIN CACHED TIME / SPECULATIVE TIME PER PROMPT. SOURCE: RESULTS/BENCHMARKS/SPECULATIVE_20261108_113154.JSON, OUR MEDIANS.

k	accept.	tok./verify	speed-up	range	draft share
1	0.974	1.97	0.981	0.775–1.134	42.1%
2	0.750	2.33	0.923	0.692–1.062	58.3%
4	0.694	3.60	0.928	0.807–1.036	72.3%
8	0.500	4.67	0.735	0.543–0.877	83.5%

rises from 42.1% to 83.5%; the median speed-up is below 1 for every k (0.981, 0.923, 0.928, 0.735), and at $k = 8$ no prompt was faster than plain decoding. At $k = 1$ one draft step costs about 0.73 of a target verify step (0.421/0.579), far above the 0.15 parameter ratio of 0.6B to 4B; this is plausible because the draft has 28 layers against 36 and batch-size-1 decoding on a T4 is likely dominated by per-layer kernel launches. A back-of-envelope estimate of 1.97/1.73, about 1.14 times, before the extra draft prefill is consistent with the observed 0.98. The answers are short (15 to 38 tokens): plain decoding of a validation prompt takes a median 1.037 s, 0.177 s of it prefill, leaving few iterations to amortise any overhead. On synthetic inputs the picture is similar at 128 and 256 input tokens (speed-up 0.40 to 1.145), but at 512 input tokens the acceptance rate is 1.0 for most rows and the speed-up reaches 1.288 times ($k = 8$, 128 new tokens); we suspect the tiled synthetic prompt yields a repetitive continuation that the draft copies easily, but did not inspect the tokens.

fp16 near-tie. 84 of the 85 result rows produce the same tokens as plain decoding. The exception (256 input tokens, 128 new tokens, $k = 1$) first differs at step 64 with a maximum logit gap of 0.0234 and a plain top-1/top-2 margin of exactly 0.0: the two best logits were tied in fp16 and the chosen token

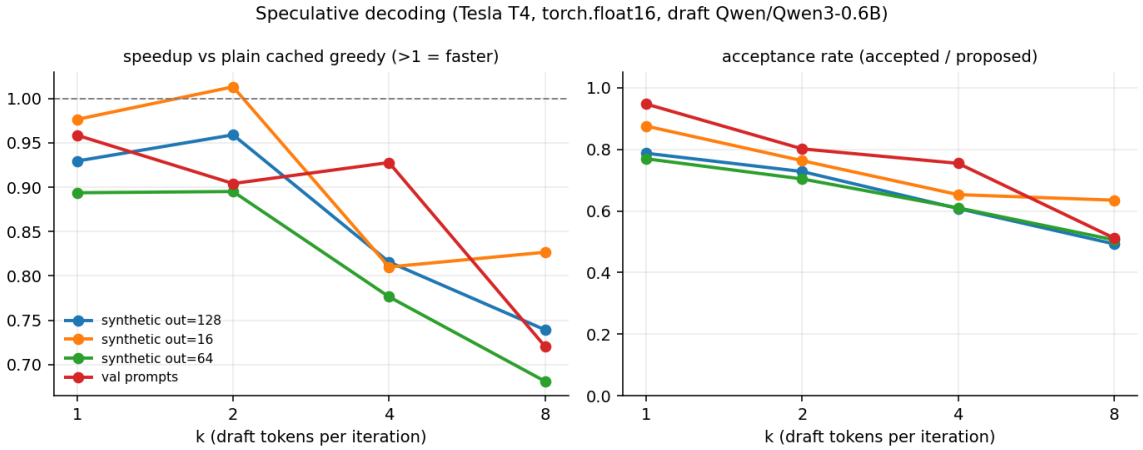


Fig. 9. Speed-up vs. plain cached greedy decoding and acceptance rate vs. k. The curves are arithmetic means over the runs of each group (synthetic: over the three input lengths; validation: over the 8 prompts), so they differ from the medians of Table IX. Source: `results/benchmarks/speculative_20261108_113154.png`.

depends on rounding. We read this as a tie, consistent with the 84 identical rows, but did not isolate the cause.

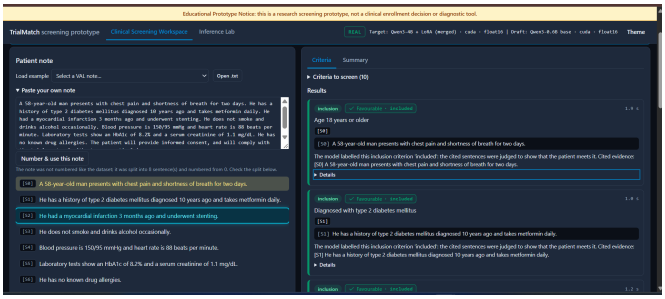
F. Application

Fig. 10 shows the workspace. In (a) a pasted, unnumbered note was split into 8 sentences numbered $[S0]\ldots[S7]$ and 10 criteria were screened; the cards show the label badge, the cited sentence and the latency (1.9 s, 1.0 s and 1.2 s for the first three cards with the KV-cache engine, read from the screenshot), and clicking a cited pill highlights the sentence in the note. The Details drawer (kept in `demo/04_details_drawer.png`) shows the raw JSON answer, parse status, engine, latency 1.85 s and 15 generated tokens. In (b) the summary reports "Undetermined: more information needed" with 0 decisive-negative, 1 unresolved, 1 not applicable and 8 favourable criteria, together with the rule and the statement that the outcome has not been validated against expert verdicts; the one unresolved criterion is the body-mass-index inclusion, labelled NEI. In (c), screening a validation note (sigir-201427) with the speculative engine took 22 s for 16 criteria, about 1.4 s per criterion, and the first card shows the "missing information" flag for an NEI answer. In (d) the exported report records mode, model, device, engine, git commit, adapter path and run id, so a result can be traced to a run; reports are also saved to `outputs/reports/`. The four views together show that the whole path (note, criteria, model, cards, summary, export) works in real mode on the T4, which is also the path used for notes and criteria that are not in the dataset.

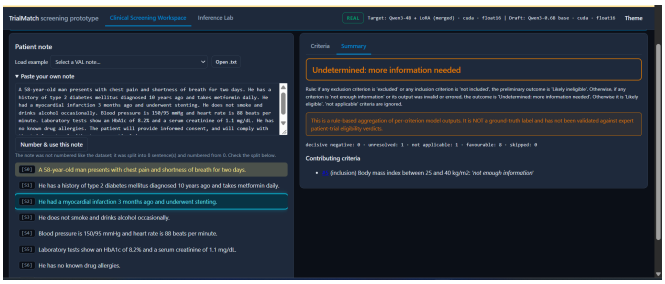
Fig. 11 shows the Inference Lab. The saved cards in (a) reproduce the speculative benchmark file for 512 input and 64 output tokens: plain 3.1 decode tok/s, KV-cached 12.2 tok/s and speculative speed-ups of 0.98, 1.05, 1.12 and 1.19 for $k = 1, 2, 4, 8$ (the file has 0.983, 1.046, 1.116 and 1.187), all with identical tokens; they are labelled "measured on Colab Tesla T4" with the date and git commit so they cannot be mistaken for live numbers. In (b) the tiled-attention visualiser shows the block grid for 32 positions with block size 8: 6 of the 16 blocks are skipped by the causal mask, and the table for query row 20 lists the running maximum, rescale factor and normaliser per key block; the footer reports a maximum difference of 2.38×10^{-7} between tiled and reference attention and 5.38 ms against 37.19 ms on a CPU with a 16-dimensional head, a small illustrative size that says nothing about the T4 results of Table VIII. In (c) a live KV-cached run on the current note (prompt 323 tokens, at most 24 new tokens) gave a prefill of 0.236 s and 17.13 decode tokens per second with a peak of 8,889 MiB; 17.13 tokens per second is 58 ms per token, inside the 56 to 87 ms per token that the saved cached runs show for validation prompts. The speculative panel was left empty in that screenshot, so the accepted/corrected token colouring is not shown in this report.

VI. Limitations

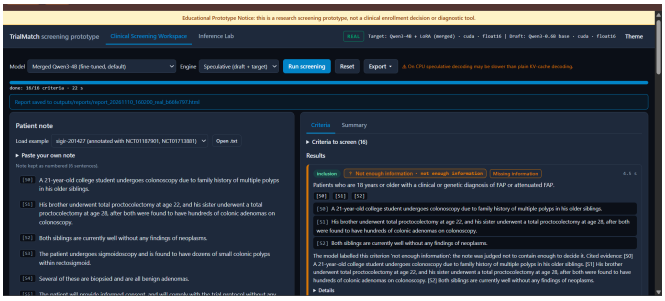
- **Single seed, single run.** One training run (seed 42) and one setting; no confidence intervals.
- **Small test set.** 8 patients and 206 rows; differences of a few rows are within noise, and rows of one patient are correlated.
- **No "excluded" rows in test.** The class has zero support on test and only 2 rows on validation.
- **Merged vs. 4-bit model.** Validation used the 4-bit-base model (0.748); test and benchmarks used the merged fp16 model (0.733 on validation).
- **CPU vs. T4.** All timings are from one Colab T4. The app was tested on CPU with fake or tiny models; the real 4B model on CPU was not exercised by the assistant, and the README's CPU time per criterion (1 to 3 minutes) is a reasoned estimate. CPU numerics can differ from T4 fp16.
- **fp16 near-ties.** One KV-cache cell (step 103, logit gap 0.0293) and one speculative row (step 64, margin 0.0) differ from the reference.



(a) Evidence sync: clicked pill highlights sentence S2



(b) Summary tab: rule text, “not a ground-truth label”, counts



(c) Speculative engine: 16 criteria screened in 22 s



(d) Exported HTML report (browser chrome cropped)

Fig. 10. Screening workspace in real mode (Qwen3-4B + LoRA merged, cuda, float16). Screenshots from the author’s sessions of 2026-11-10; numbers quoted in the text are read from them.

- **Evidence weaknesses.** Exclusion evidence micro-F1 is 0.314 on test; the model over-cites, under-cites and sometimes cites the consent sentence. About 60% of gold evidence lists are empty, which inflates metrics that include them; both views are reported.
- **Missing information treated as absence.** For exclusion criteria the model often answers “not excluded” when the note does not mention the criterion, and it did not apply a six-month window to a three-month-old event in our app test.
- **Candidate comparison.** Only Qwen3-4B was fine-tuned; the Llama alternative could not be measured (gated) and its architecture figures come from the model cards, not from our measurement; the draft’s parameter count differs between the hub metadata and the loaded model.
- **Educational implementations.** Speculative decoding is batch size 1 with an untuned draft and gave no speed-up on real prompts; tiled attention is plain PyTorch, not used inside the model, and slower than fused kernels.
- **Educational prototype.** The preliminary outcome is a rule over criterion labels, not validated against patient-level verdicts; the system must not be used for clinical decisions. The screenshots come from the author’s sessions, all in dark theme.

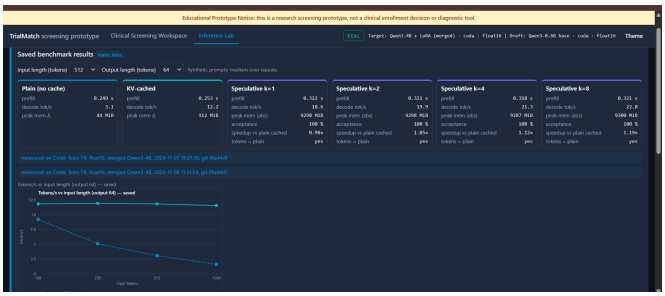
VII. Conclusion

QLoRA fine-tuning of Qwen3-4B on 603 patient-grouped rows improves criterion-level accuracy from 0.665 to 0.767 and macro-F1 over present classes from 0.523 to 0.726 on a held-out test split, and improves evidence F1, but the model tends to answer “not excluded” where annotators saw insufficient information, and evidence for exclusion criteria stays weak. The hand-written KV cache gives a 3.0 times speed-up on real prompts and up to 7.1 times at 1024 tokens; tiled attention cuts peak memory by up to 180 times at 4096 tokens but not time; speculative decoding with an untuned small draft did not help on these short answers. The browser application screened 16 criteria in 22 s on a T4 and records every run. Next steps are a fine-tuned draft, several seeds and a larger or patient-level evaluation. All numbers come from the repository’s result files or are read from the screenshots; all causal explanations are hypotheses.

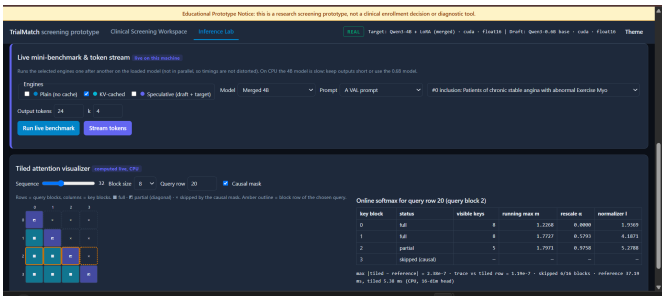
Appendix A
AI Use

Claude Code (coding assistant). Used in several sessions from 2026-11-06 to 2026-11-10 (`docs/ai_use_log.md`) to write code and tests for: data download, inspection, grouped split and leakage checks; prompt formatter, output schema and parser; model measurement scripts; LoRA/QLoRA training and the evaluation runner; metrics, baselines and error-analysis export; the merged-model loader, KV-cache decoder and benchmark; tiled attention and its benchmark; speculative decoder and benchmark; the browser application; final test-metric flags and app fixes; and, in the last sessions, drafts of the notes files, the error analysis, this paper (including the diagrams and algorithm boxes) and the README.

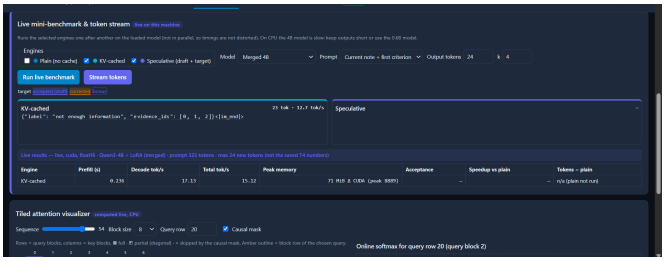
What the author verified (from the AI use log). Detected label, evidence and leakage columns against the raw data; label sums against row counts (an earlier summary omitted NEI



(a) Saved T4 benchmarks



(b) Live benchmark controls and tiled-attention visualiser



(c) Live KV-cached run on the current note

Fig. 11. Inference Lab. (a) saved results for 512 input and 64 output tokens; (b) controls of the live mini-benchmark and the tiled-attention visualiser (sequence 32, block 8, query row 20); (c) a live run with the KV-cached engine. Numbers quoted in the text are read from the screenshots.

rows); a gold round trip over all rows; a KV-cache test value by hand ($36 \times 8 \times 128 \times 1024 \times 2 \times 2 = 150{,}994{,}944$ bytes); that no weights were downloaded and the test split was not read; that gated Llama repositories are reported as skipped; hand-computed metric test values and an sklearn cross-check; decoder and cache invariants through tests on tiny random models; that app tests used fake backends. Not exercised by the assistant: GPU training and evaluation (run by the author on Colab), the real 4B model on a CPU, and the UI in a browser (the author took the screenshots).

Claude chat (conversational assistant). Used, according to the author, for concept explanation and for refining the prompts given to Claude Code.

Responsibility. The notes, error analysis and this paper were drafted by the assistant from repository files and the author's screenshots; every number carries its source file (in the comments of this file) and every causal explanation is marked as a hypothesis; the author must review and rewrite the prose. Six references use BibTeX supplied by the author; the entries for the Qwen3 report, LoRA, Attention Is All You Need, the online softmax and speculative decoding by Leviathan et al. were completed by the assistant from memory and should be checked.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, no. 1, p. 9074, 2024.

[2] A. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in Neural Information Processing Systems*, vol. 36, pp. 10 088–10 115, 2023.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[7] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[8] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 4895–4901.

[9] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[10] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*, 2023.

[11] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.